TECHNICAL REPORT & ARCHITECTURAL REVIEW

Migration Analysis: supabase/migrations/20261110_create_payments.sql

Date: November 10, 2026 | Status: Read-Only Database Assessment Complete | Target: B2B LINK OÜ (b2blink.eu)

EXECUTIVE AUDIT SUMMARY

A live, non-destructive audit was performed comparing the proposed migration (20261110_create_payments.sql) against the active Supabase PostgreSQL database. Key determinations:

• Migration Status: public.payments ALREADY EXISTS in the live database with identical 16 columns, active foreign keys, text-length checks, and performance indexes.

• Idempotency & Safe No-Op: Because the table exists, executing the proposed CREATE TABLE IF NOT EXISTS migration again will succeed with 0 errors but will silently do nothing.

• Security & RLS: RLS is active and forced. Zero public policies exist; client anon and auth queries return 0 rows. Access is strictly confined to elevated server-side Service Role handlers.

• Future Architecture: Approved to decouple the financial ledger from future monthly subscriptions, specifying Day 1/3/5 retry backoffs, period-end cancellations, and prorated upgrades.

1. CORE AUDIT FINDINGS AT A GLANCE

Table Status	Active in Live Database (16 Columns, 5 Indexes, 6 Check Constraints)
RLS & Force RLS	ENABLED & FORCED (Zero public policies; 100% blocked on client)
Migration Impact	CREATE TABLE IF NOT EXISTS is a safe no-op; leaves schema unchanged
updated_at Trigger	No DB trigger; route handler explicitly passes updated_at in queries
Cross-Tenant Access	Protected; API handlers strictly enforce requireAdmin and requireUser
Subscriptions	Requires future public.subscriptions table to decouple ledger from recurring state

2. EXISTING PUBLIC.PAYMENTS SCHEMA SPECIFICATION

Inspected directly via PostgREST OpenAPI catalog and verified through check constraint probes.

COLUMN	POSTGRES TYPE	NULL	DEFAULT	CONSTRAINTS / ENFORCED RULES
id	text	NO	—	PRIMARY KEY (e.g. PAY-1001)
company_id	uuid	YES	—	FK -> public.companies(id) ON DELETE SET NULL
company_name	text	NO	—	char_length(company_name) <= 255
contact_email	text	NO	—	char_length(contact_email) <= 255
plan_or_service	text	NO	—	char_length(plan_or_service) <= 100
amount_gbp	numeric(10,2)	NO	—	amount_gbp >= 0
payment_method	text	NO	—	char_length(payment_method) <= 100
wise_reference	text	YES	—	IS NULL OR char_length <= 150
transaction_id	text	YES	—	IS NULL OR char_length <= 150
status	text	NO	'Completed'	Completed | Pending Verification | Failed | Refunded
invoice_number	text	NO	—	char_length(invoice_number) <= 100

notes	text	YES	—	IS NULL OR char_length <= 2000

recorded_by	uuid	YES	—	FK -> auth.users(id) ON DELETE SET NULL

created_at	timestamptz	NO	now()	Generated automatically on insertion

completed_at

timestamptz

YES

Populated when status transitions to Completed

updated_at

timestamptz

NO

now()

	No trigger; manually passed in route handlers

3. MIGRATION COMPARISON & IDEMPOTENCY ASSESSMENT

Comparison between supabase/migrations/20261110_create_payments.sql and live database:
1. Existing Columns: All 16 columns declared in the migration exist with identical types, nullability, and default expressions.
2. Check Constraints: payments_status_check, payments_amount_gbp_check, and char_length constraints are active.
3. Indexes: idx_payments_company_id, idx_payments_status, idx_payments_created_at, and idx_payments_invoice_number are active.
4. CREATE TABLE IF NOT EXISTS Behavior: Because public.payments already exists, running the migration will skip table creation entirely. It will not fail, but it will silently leave the schema 100% unchanged.
5. Crucial Migration Rule: Any future alterations (such as adding subscription_id) must NOT be written inside CREATE TABLE IF NOT EXISTS. They must be authored in a new versioned migration using ALTER TABLE public.payments ADD COLUMN IF NOT EXISTS ...

4. ACCESS CONTROL & SECURITY REVIEW

Client Anon & Auth Access
No public RLS policies exist. Both anonymous and authenticated users receive empty results (0 rows) if querying /rest/v1/payments directly. Direct client tampering is impossible.

FORCE ROW LEVEL SECURITY
Enforced via ALTER TABLE payments FORCE ROW LEVEL SECURITY. Closes standard PostgreSQL table-owner bypasses. Only the Service Role (BYPASSRLS) can access records.

Admin API Handler Isolation
/api/admin/payments is strictly protected by requireAdmin() across GET, POST, and DELETE. Regular members and unauthenticated callers receive HTTP 401 or 403.

Account Deletion Cascade Isolation
/api/auth/delete-account derives company_id strictly from session (auth.company.id). Deletes payments using .eq("company_id", tId). Cross-company deletion is prevented.

Member Billing Isolation
/dashboard/billing does not expose cross-company payments. To allow members to view their own invoices in the future, an authenticated, company-scoped endpoint should be added.

5. FUTURE MONTHLY SUBSCRIPTION ARCHITECTURE

Architectural specification decoupling immutable financial ledger from recurring subscription lifecycle.

A. Decoupled Ledger vs. State Machine
Keep public.payments as an append-only transaction ledger (receipts, amounts, gateway refs). Create a dedicated public.subscriptions table to track monthly billing periods, renewal dates, retry states, and cancellation flags.

B. Day 1, 3, 5 Retry Schedule with Paid Access
When a renewal payment fails on Day 0:
• Record Failed in payments ledger. Set subscription status to past_due, retry_count = 1, next_retry_at = +24h.
• Paid access (Silver/Gold) remains active during retries.
• Automated cron retries on Day 1 (retry_count = 2) and Day 3 (retry_count = 3).
• If Day 5 retry fails: Immediately transition subscription to expired/canceled and revert company.tier = "bronze". No extra grace period.

C. Cancellation at Period End

When a user cancels, set cancel_at_period_end = true. The company retains Silver/Gold access until current_period_end. When the period ends, the cron sweep transitions status to canceled and sets tier = "bronze".

D. Immediate Prorated Silver-to-Gold Upgrade

Upgrades apply immediately. Prorated cost = Remaining Days * ((Gold Price / 30) - (Silver Price / 30)). Charge customer immediately, record Completed in payments, and switch company.tier = "gold".

E. Undecided Pricing Principle

Do not bake hardcoded prices (£50, £150) into database check constraints or defaults. Prices must remain dynamic in gateway plans or environment configuration, ensuring zero SQL migrations for price updates.

6. FUTURE DATABASE SCHEMA BLUEPRINT

Reference DDL to be applied when monthly subscription feature is approved for implementation:

```
CREATE TABLE public.subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES public.companies(id) ON DELETE CASCADE,
  tier TEXT NOT NULL CHECK (tier IN ('silver', 'gold')),
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'past_due', 'canceled', 'expired',
'incomplete')),
  current_period_start TIMESTAMPTZ NOT NULL DEFAULT now(),
  current_period_end TIMESTAMPTZ NOT NULL,
  cancel_at_period_end BOOLEAN NOT NULL DEFAULT false,
  canceled_at TIMESTAMPTZ,
  retry_count INTEGER NOT NULL DEFAULT 0 CHECK (retry_count BETWEEN 0 AND 3),
  next_retry_at TIMESTAMPTZ,
  gateway_customer_id TEXT,
  gateway_subscription_id TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

7. AFFECTED DATABASE OBJECTS & APPLICATION FILES

DB Table	**public.payments**	Payment transaction ledger (Already active with RLS)
DB Table	**public.companies**	Company entity; stores current member tier and approval_status
DB (Future)	**public.subscriptions**	Future state machine table for recurring cycles and retries
Migration	**supabase/migrations/20261110_create_payments.sql**	DDL baseline on branch (safe no-op if executed again)
TypeScript	**src/types/database.ts**	Database type mappings for Payment interface
API Route	**src/app/api/admin/payments/route.ts**	Admin ledger management (GET, POST record, POST verify, DELETE)
API Route	**src/app/api/billing/activate-tier/route.ts**	Sandbox membership tier activator (PAYMENT_SANDBOX_MODE)
API Route	**src/app/api/auth/delete-account/route.ts**	Cascades payment deletion scoped to caller company
UI Page	**src/app/site-admin/payments/page.tsx**	Administrative payment desk and transaction review
UI Page	**src/app/dashboard/billing/page.tsx**	Member billing dashboard and plan management